

Sommario

I) Cosa occorre per realizzare Diffie-Hellman?

1) Un algoritmo efficiente per calcolare $a^b \bmod q$

Algoritmo iterativo per calcolare $a^b \bmod q$

Proprietà B dell'aritmetica modulare

Algoritmo in codice

2) Un algoritmo efficiente per generare q primo

Teorema dell'infinità dei numeri primi (Euclide)

Teorema dei numeri primi

Test di Miller-Rabin - basi teoriche

Identità di Bezout o Teorema

Teorema di Eulero

Dimostrazione del Teorema di Eulero

Proprietà III dell'aritmetica modulare

Piccolo Teorema di Fermat

Principio Fondamentale

Dimostrazione del Principio Fondamentale

Lemma di Euclide

Conseguenza del Principio Fondamentale

Dimostrazione della Conseguenza al Principio Fondamentale (slides)

Dimostrazione della Conseguenza al Principio Fondamentale (alternativa)

Algoritmo di generazione primi grandi

Come funziona Miller-Rabin?

{Teorema Fondamentale dell'aritmetica}

Algoritmo per il test di Miller-Rabin

3) Un algoritmo efficiente per generare una radice primitiva di q

Algoritmo per trovare una radice primitiva di q

Dimostrazione algoritmo radice primitiva

II) Come funziona Diffie-Hellman?

1) Fasi di Diffie-Hellman

2) Problemi di Diffie-Hellman

Scambio di chiavi di Diffie-Hellman

1) Cosa occorre per realizzare Diffie-Hellman?

1) Un algoritmo efficiente per calcolare $a^b \bmod q$

Ovvero un algoritmo di esponenziazione modulare. Per evitare la complessità lineare della soluzione “ingenua”, si usa un

Algoritmo iterativo per calcolare $a^b \bmod q$ (in parole)

che agisce in questo modo:

b viene interpretato come la somma di potenze di due. Ad esempio, se $b = 22$, lo si interpreta come $16 + 4 + 2$, da un punto di vista pratico cioè se ne considera il bit vector corrispondente 10110 . Per ciascun bit posto a 1 nel bit vector, in posizione i , si eleva a alla 2^i . Si ottiene una produttoria in cui contribuiscono solo i termini a^{2^i} elevati all'indice dei bit posti a 1 nel bit vector. Nell'esempio, abbiamo bit a 1 nella posizione 4, 2, e 1, quindi:

$$\text{risultato} = (a^{2^4 * 4} * a^{2^2 * 2} * a^{2^1 * 1}) \bmod q$$

Ci viene in aiuto la

Proprietà B dell'aritmetica modulare:

$$(a^b) \bmod M = (a \bmod M)^b \bmod M$$

Infatti (dimostrazione)

è un caso particolare della Proprietà A, vedi capitolo RSA

Perciò, invece di moltiplicare tutti i termini della produttoria così come sono, posso prima applicarvi l'operazione di modulo, semplificando molto la computazione:

$$\text{risultato: } [(a^{2^4 * 4} \bmod q) * (a^{2^2 * 2} \bmod q) * (a^{2^1 * 1} \bmod q)] \bmod q$$

Da un punto di vista del codice, otterremo un

Algoritmo iterativo per calcolare $a^b \bmod q$ (in codice)

di complessità $O(\log n)$:

```
res = 1;
a2i = a; //ovvero, a elevato alla  $2^0 \bmod q$ 
for(i = 0; i < b.length; i++){
    if(b[i] == 1){
        res = (res * a2i) % q;
    }
    a2i = (a2i * a2i) % q; //  $a^{2^i} * a^{2^i} = a^{2^{i+1}}$ 
}
return res;
```

Esiste anche una versione ricorsiva, non utilizzata in quanto inefficiente

2) Un algoritmo efficiente per generare q primo

Un metodo banale potrebbe essere quello di generare un numero a caso e poi controllare che sia primo. Per capire se ciò è implementabile praticamente occorre chiedersi quanto sia facile che un numero casuale sia primo. Prima di tutto sappiamo che i numeri primi sono infiniti grazie al

TEOREMA DELL'INFINITÀ DEI NUMERI PRIMI (Euclide).

Ci sono infiniti numeri primi.

Dimostrazione:

Supponiamo che p sia l'ultimo numero primo e che $q = 2 \cdot 3 \cdot 5 \cdot \dots \cdot p$ sia il prodotto di numeri primi fino a p. Per il **Teorema fondamentale dell'aritmetica**, ogni numero > 1 , o è primo, o può essere espresso come prodotto di numeri primi, dunque vediamo i due casi:

- se $q + 1$ è primo, allora evidentemente p non era l'ultimo
- se $q + 1$ NON è primo, allora sarà divisibile per un qualche numero r primo; ma dividendolo per un qualsiasi numero primo minore o uguale a p, otterrò sempre resto 1 poiché q è il prodotto di tutti i numeri primi fino a p. Quindi r non può che essere maggiore di p, e ne consegue che p non era l'ultimo numero primo.

Abbiamo dunque la certezza che i numeri primi sono infiniti; ma come sono distribuiti su $\mathbb{N}$? Se sono molto “sparsi”, potrebbe non convenire sceglierne uno casualmente. A questo risponde il

TEOREMA DEI NUMERI PRIMI

sia $\pi(x)$ il numero di primi minori o uguali all'intero x . Allora

$$\pi(x) \sim \frac{x}{\log x}.$$

ovvero

$$\lim_{x \rightarrow \infty} \frac{\pi(x)}{\left[\frac{x}{\log(x)} \right]} = 1,$$

Il teorema cioè dice che $x / \log(x)$ approssima $\pi(x)$ (l'errore tende a 0 all'aumentare di x).

In crittografia occorrono numeri primi molto grandi!

Allora considero:

i numeri di 100 cifre, che sono 10^{100} ;

i numeri primi di 100 cifre sono:

$$\pi(10^{100}) - \pi(10^{99}) = [10^{100} / \ln(10^{100})] - [10^{99} / \ln(10^{99})] \approx 3.9 * 10^{97}$$

Perciò, in media avrò un numero primo ogni $10^{100}/3.9*10^{97}$, ovvero ne troverò uno ogni 256 tentativi.

Questo risultato è ulteriormente ottimizzabile evitando i numeri pari e i multipli di 3, 5, 7 e 11 per i quali ci sono controlli immediati.

Allora per generare un numero primo dovrei iterare il mio algoritmo banale un centinaio di volte; tuttavia per controllare se un numero è primo devo ogni volta fare molte divisioni, quindi non è ancora abbastanza efficiente. Si usa quindi un metodo probabilistico: l'algoritmo si ripete solo un numero T di volte, abbastanza da rendere molto probabile di aver trovato un numero primo:

$$P(M \text{ non primo}) = 2^{-T}, \quad P(M \text{ primo}) = 1 - 2^{-T}.$$

Per verificare che M sia primo, devo usare un test adeguato: il

TEST DI MILLER-RABIN - Basi Teoriche

Piccolo preambolo:

IDENTITÀ DI BEZOUT (nelle slides chiamato TEOREMA I):

se a e b non sono nulli, $\exists (x,y) ax+by=\text{MCD}(a,b)$, ovvero il Massimo Comun Divisore di due interi è una combinazione lineare tra i due, con opportuni coefficienti interi.

Dimostrazione dell'Identità di Bezout (o Teorema I):

Per dimostrarlo basta far vedere che tutti i resti delle divisioni successive per trovare l'MCD(a, b) si possono scrivere come combinazioni di a e b , ovvero $\exists (x_i, y_i) ax_i + by_i = r_i$.

Dall'**Algoritmo di Euclide per l'MCD** (spiegato [qui](#)), sappiamo che $r_0 = a$, $r_1 = b$, e in generale

$$(a) \quad r_k = q_{k+1} * r_{k+1} + r_{k+2},$$

vale a dire il resto ottenuto in una certa iterazione k è pari al prodotto di quoziente e resto dell'iterazione precedente, più il resto dell'iterazione ancora precedente, da cui posso ricavare

$$(b) \quad r_{k+2} = r_k - q_{k+1} * r_{k+1}$$

Da questo possiamo ricavare che:

I resti possono essere scritti come combinazioni lineari? (caso base)

- $r_0 = a = 1*a + 0*b$ OK!
- $r_1 = b = 0*a + 1*b$ OK!
- $r_2 = r_0 - q_1 * r_1 = a - q_1 * b$ OK!

Supponiamo che questo valga fino a $i-1$, e quindi anche per $i-2$.

Dimostriamo che se vale per $i-1$ allora vale anche per i (induzione):.

Per (b), vale che:

$$r_i = r_{i-2} - q_{i-1} * r_{i-1} = (a*x_{i-2} + b*y_{i-2}) - q_{i-1} * (a*x_{i-1} + b*y_{i-1})$$

semplificando:

$$\begin{aligned} & a*x_{i-2} + b*y_{i-2} - q_{i-1} * a*x_{i-1} - q_{i-1} * b*y_{i-1} \\ & a*(x_{i-2} - q_{i-1} * x_{i-1}) + b*(y_{i-2} - q_{i-1} * y_{i-1}) \\ & \quad \uparrow \quad \quad \quad \uparrow \\ & \quad x_1 \quad \quad \quad y_1 \end{aligned}$$

C.V.D.

Torniamo a Miller-Rabin. Questo test utilizza due teoremi:

1. **Piccolo Teorema di Fermat** (dimostrato come corollario al Teorema di Eulero)

se M è primo e $a < M$, allora $a^{M-1} \bmod M = 1$

2. Conseguenza del Principio Fondamentale

se M è primo e $x^2 \bmod M = 1$, allora $x = 1$ o $x = M - 1$

Per poter dimostrare il **P.T.d.F.**, dimostriamo prima il

TEOREMA DI EULERO

Definizione: $\Phi(n)$ è il numero di interi minori di n che sono relativamente primi con n .

Il teorema dice che: se M , a relativamente primi $a^{\Phi(M)} \bmod M = 1$.

Dimostrazione del Teorema di Eulero

In questa dimostrazione utilizzeremo la

Proprietà III dell'aritmetica modulare:

se n, a relativamente primi, allora

$$ab \equiv ac \pmod{n} \rightarrow b \equiv c \pmod{n}$$

In altre parole, posso dividere per a e semplificare.

Infatti (dimostrazione):

Per **identità di Bezout**, se $\text{MCD}(a, M) = 1$, allora $\exists(x, y)$ tali che $ax + ny = 1$. Quindi ho un semplice procedimento algebrico:

$$(b - c) * (ax + ny) = b - c$$

$$(b - c) * ax + (b - c) * ny = b - c$$

$$(ab - ac) * x + (b - c) * ny = b - c$$

per def. di congruenza: $ab \equiv ac \pmod{n} \Rightarrow ab - ac = kn$, quindi:

$$\mathbf{b - c} = (ab - ac) * x + (b - c) * ny =$$

$$= knx + (b - c) * ny = \mathbf{[kx + (b - c) * y] * n},$$

ovvero $(b - c)$ è multiplo di n , e per def. di congruenza:

$$b - c \equiv 0 \pmod{n} \Rightarrow b \equiv c \pmod{n}$$

c.v.d.

- Immaginiamo un insieme

$$R = \{x_1, x_2, \dots, x_{\Phi(M)}\}$$

ovvero l'insieme di numeri minori di M relativamente primi con M .

- Se moltiplico ciascun membro di R per a , e di ciascun membro faccio l'operazione modulo M , ottengo un altro insieme che chiamo

$$S = \{ax_1 \bmod M, ax_2 \bmod M, \dots, ax_{\Phi(M)} \bmod M\}$$

Ogni $ax_i \bmod M$ è ancora coprimo con M : infatti, ax_i è prodotto di due numeri coprimi con M , e perciò ogni numero primo p che dividesse ax_i dividerebbe o a o x_i , e quindi se dividesse anche M vorrebbe dire che o a o x_i non sarebbero più coprimi con M , il che contraddice l'ipotesi.

- Ci domandiamo: S e R sono lo stesso insieme?
In effetti, abbiamo R che è un insieme di numeri coprimi con M fino a M con cardinalità $\Phi(M)$, e di S sappiamo che ha anch'esso cardinalità $\Phi(M)$ e contiene tutti numeri coprimi con M . Inoltre l'operazione modulo ci garantisce che i membri di S rimarranno minori di M . Dunque

- S è incluso in R perché tutti gli $ax_i \bmod M$ sono coprimi con M

L'unico modo in cui i due insiemi potrebbero differire, quindi, è che in S ci siano delle ripetizioni, ma:

- R è incluso in S poiché, per la **Proprietà III**:

$$\text{se } ax_i \bmod M = ax_j \bmod M \Rightarrow x_i = x_j$$

- Abbiamo dunque dimostrato che $R = S$, e ciò ci consente di ottenere un semplice risultato algebrico:

$$\{x_1, x_2, \dots, x_{\Phi(M)}\} = \{ax_1 \bmod M, ax_2 \bmod M, \dots, ax_{\Phi(M)} \bmod M\} // R \text{ e } S \text{ uguali}$$

//produttorie uguali e quindi congruenti:

$$(x_1 * x_2 * \dots * x_{\Phi(M)}) \bmod M = (ax_1 \bmod M * ax_2 \bmod M * \dots * ax_{\Phi(M)} \bmod M) \bmod M$$

//riscrivo secondo membro in base alla Proprietà I:

$$(x_1 * x_2 * \dots * x_{\Phi(M)}) \bmod M = (ax_1 * ax_2 * \dots * ax_{\Phi(M)}) \bmod M$$

//raccolgo a al secondo membro; ce n'è esattamente $\Phi(M)$:

$$(x_1 * x_2 * \dots * x_{\Phi(M)}) \bmod M = a^{\Phi(M)}(x_1 * x_2 * \dots * x_{\Phi(M)}) \bmod M$$

//la produttoria $(x_1 * x_2 * \dots * x_{\phi(M)})$ è coprima con M in quanto ogni x_i è coprimo con M ; quindi posso usare la Proprietà III e semplificare:

$$1 \bmod M = a^{\phi(M)} \bmod M$$

//quindi

$$a^{\phi(M)} \bmod M = 1$$

//C.V.D.

PICCOLO TEOREMA DI FERMAT

se M è primo e $a < M$, allora $a^{M-1} \bmod M = 1$

Dimostrazione

Il P.T.d.F. è un corollario al Teorema di Eulero.

Vediamo subito che “se M è primo e $a < M$, allora $a^{M-1} \bmod M = 1$ ” è soltanto un caso particolare del teorema di Eulero: in effetti, se M è primo, $\phi(M) = M - 1$.

Infatti ogni numero prima di M è coprimo con M , poiché M è primo. Quindi $a^{M-1} \bmod M = a^{\phi(M)} \bmod M = 1$ C.V.D.

Per poter dimostrare la **Conseguenza al Principio Fondamentale**, dimostriamo prima il

PRINCIPIO FONDAMENTALE

se esistono x e y tali che $x^2 \equiv y^2 \pmod{n}$, e $\neg(x \equiv \pm y \pmod{n})$, allora n non è primo.

Idea: data questa ipotesi, $\text{MCD}(x-y, n)$ non può che essere diverso sia da 1 sia da n , il che equivale a dire, che esiste un divisore di n diverso da 1 e n , ovvero n non è primo

Dimostrazione del Principio Fondamentale

In questa dimostrazione utilizzeremo il

Lemma di Euclide

secondo cui se $n \mid XY$ e $\text{MCD}(n, X)$, allora $n \mid Y$

Infatti(dimostrazione):

- n non divide X
- deve esistere un r tale che $r * n = x*y$ poiché n divide $x*y$
- per **identità di Bezout**, se $\text{MCD}(n, X) = 1$, $\exists(x,y)$ t. c. $n*x + a*y = 1$
 - moltiplico tutto per b : $b*n*x + a*b*y = b$
- so che $a*b = r * n$ per un certo r , quindi $b*n*x + r*n*y = b$
 - raccolgo: $n*(b*x + r*y) = b$, ovvero $n*$ qualcosa $= b$,
ovvero n divide b ,
c.v.d

sia $d = \text{MCD}(x-y, n)$.

- d non può essere uguale a n perché, se $d=n$, allora $(x-y)$ è divisibile per n , ovvero $(x-y) \equiv 0 \pmod{n}$, e quindi $x \equiv y \pmod{n}$, il che **contraddice l'ipotesi**.

Infatti (nella dimostrazione seguente, il segno $/$ indica l'operazione di divisione intera, mentre $|$ indica "divide"):

$$\begin{aligned}(x-y) \equiv 0 \pmod{n} &\Rightarrow n \mid (x-y) \Rightarrow n*k = (x-y) \Rightarrow x = k*n + y, \\ y = -k*n + x &\Rightarrow y / n = (-k*n + k*n + y) / n = y; x / n = (k*n \\ &+ y) / n = y \Rightarrow x \equiv y \pmod{n} \text{ che } \mathbf{contraddice}\end{aligned}$$

l'ipotesi $\neg(x \equiv + y \pmod{n})$

- d non può essere uguale a 1 perché, se $d=1$:
sappiamo che

$$\begin{aligned}x^2 \equiv y^2 \pmod{n} &\Rightarrow x^2 - y^2 \equiv 0 \pmod{n} \Rightarrow n \mid x^2 - y^2 \Rightarrow \\ &n \mid (x-y)(x+y).\end{aligned}$$

Ma per quanto detto sopra, n non può dividere $x-y$.

Allora, per il **Lemma di Euclide**, dovrà dividere

necessariamente $x+y$; ma questo **contraddice l'ipotesi**

$\neg(x \equiv - y \pmod{n})$.

Infatti(nella dimostrazione seguente, il segno $/$ indica l'operazione di divisione intera, mentre $|$ indica "divide"):

$$n \mid x + y \Rightarrow x + y = k*n \Rightarrow x = k*n - y, y = k*n - x \Rightarrow$$

$$\begin{aligned}
 -y / n &= -(k*n - x) / n = -(k*n - k*n + y) / n = \\
 (-k*n + k*n - y) / n &= -y; \quad x / n = (k*n - y) / n = -y \Rightarrow \\
 x &\equiv y \pmod{n} \text{ che } \mathbf{contraddice} \\
 \mathbf{l'ipotesi} \quad &\neg(x \equiv -y \pmod{n})
 \end{aligned}$$

Conclusione: siccome MCD non può essere né 1 né n, n non può essere primo, C.V.D.

Da questo Principio Fondamentale deriva la

CONSEGUENZA AL PRINCIPIO FONDAMENTALE:

(**Principio Fondamentale:** se esistono x e y tali che $x^2 \equiv y^2 \pmod{n}$, e $\neg(x \equiv \pm y \pmod{n})$, allora n non è primo; di conseguenza)
 se M è primo e $x^2 \pmod{M} = 1$, allora $x \equiv 1 \pmod{M}$ o $x \equiv M - 1 \pmod{M}$ (nelle slides c'è un errore: dice "allora $x = 1$ o $x = M - 1$ ", che è giusto solo aggiungendo la condizione che $x < M$.)

Dimostrazione della Conseguenza al Principio Fondamentale (slides):

$\exists(x, y) \text{ t.c. } x^2 \equiv y^2 \pmod{n} \wedge \neg(x \equiv \pm y \pmod{n}) \Rightarrow n \text{ non è primo}$
 sfruttando la formula logica $(P \Rightarrow Q) \Rightarrow (\neg Q \Rightarrow \neg P)$, ottengo:
 $n \text{ è primo} \Rightarrow \neg((\exists(x, y) \text{ t.c. } x^2 \equiv y^2 \pmod{n} \wedge \neg(x \equiv \pm y \pmod{n})))$
 (dove "n è primo" è la riscrittura di " $\neg$ n non è primo")
 sfruttando la formula logica $\neg(P \wedge Q) = \neg P \vee \neg Q$ ottengo:
 $n \text{ è primo} \Rightarrow (\neg(\exists(x, y) \text{ t.c. } x^2 \equiv y^2 \pmod{n}) \vee (x \equiv \pm y \pmod{n}))$
 sfruttando la formula logica $(P \Rightarrow Q \vee T) \Rightarrow P \wedge \neg Q \Rightarrow T$ ottengo:
 $n \text{ è primo e } (\exists(x, y) \text{ t.c. } x^2 \equiv y^2 \pmod{n}) \Rightarrow (x \equiv \pm y \pmod{n})$
 Perciò se M è primo e $x^2 \equiv 1 \pmod{M}$, siamo nel caso in cui $y^2 = 1$ e perciò $y = \pm 1$. Ciò **implica che $x \equiv \pm 1 \pmod{M}$** (e se $x \equiv -1 \pmod{M}$ allora $x \equiv M - 1 \pmod{M}$) //c.v.d.

Dimostrazione della Conseguenza al Principio Fondamentale (alternativa):

$$x^2 \equiv 1 \pmod{M} \Rightarrow x^2 - 1 \equiv 0 \pmod{M} \Rightarrow$$

$M \mid x^2 - 1$ ovvero $M \mid (x - 1)(x + 1)$ quindi per il **Lemma di Euclide** M deve dividere uno dei due fattori.

se $M \mid (x - 1)$ allora $x \equiv -1 \pmod{M}$

se $M \mid (x + 1)$ allora $x \equiv 1 \pmod{M}$

A questo punto possiamo finalmente definire un

Algoritmo per generare numeri primi grandi

```
1. genera M di k bit a caso
2. for (i=0; i<T; i++) //ripeti T volte
   genera a<M a caso
   if (test(a,M)) goto 1 // M non primo
3. return M // Pr(M primo) > 1 - 4-T
```

COME FUNZIONA PERÒ IL TEST DI MILLER-RABIN?

Il test(M, a) può dare due esiti:

a) M non è primo

b) M è probabilmente primo.

se $M > 2$ e primo, allora M è dispari e quindi $M - 1$ sarà pari. Perciò può essere scritto:

$M - 1 = 2^k \cdot q$, con $k > 0$ e $q \bmod 2 = 1$ (ovvero q è dispari).

{Questo deriva dal

APPROFONDIMENTO..

TEOREMA FONDAMENTALE DELL'ARITMETICA

per il quale ogni intero > 1 si può scrivere come prodotto di numeri primi.

In questo caso $M - 1$ è pari; voglio dimostrare che un pari può sempre scriversi come prodotto tra una potenza di 2 e un numero dispari.

Dimostrazione:

una potenza di 2 è sempre pari, quindi posso scriverlo come

$2^k \cdot p_1 \cdot p_2 \cdot \dots \cdot p_n$ (se uno dei fattori primi è 2, va a far parte della parte pari; una potenza di 2 moltiplicata per 2 rimane sempre una potenza di 2. Tutti gli altri saranno primi maggiori di 2, quindi dispari).

Dal momento che i fattori primi maggiori di 2 sono tutti dispari, il loro prodotto sarà dispari.

[dimostrazione:

$(2n+1) \times (2m+1) = 4nm + 2n + 2m + 1 = 2(2nm + n + m) + 1$ forma base di un numero dispari.]

Ho quindi dimostrato che un numero pari può essere scritto come potenza di 2 per un numero dispari.

}

Vedendo $M - 1$ come un numero binario, k è il numero di zeri in coda.

Dal **Piccolo Teorema di Fermat** sappiamo che:

$$a^{M-1} \bmod M = 1, \text{ quindi } a^{q \cdot 2^k} \bmod M = 1$$

Ora prendiamo in considerazione la sequenza di $a^{q \cdot 2^i} \bmod M = 1$ per ogni i da 0 a k :

$$a^q \bmod M, a^{q \cdot 2} \bmod M, a^{q \cdot 4} \bmod M, \dots, a^{q \cdot 2^k} \bmod M$$

Di cui so che l'ultimo termine è uguale a 1. Notiamo che ogni termine è radice quadrata di quello precedente, e che se $a^{q \cdot 2^k} \bmod M = 1$, ci sono due possibili casi:

I) Tutti i termini della sequenza sono uguali a 1 (ovvero tutti gli $a^{q \cdot 2^i}$ sono congrui a 1 modulo M)

II) La sequenza non parte con 1 ($a^q \bmod M \neq 1$)

In questo caso, abbiamo che a un certo punto della sequenza dovrà comparire il primo 1 (è per forza così, poiché sappiamo per certo che l'ultimo termine è congruo a 1). Poniamo che questo avvenga con una certa i :

$$a^{q \cdot 2^i} \bmod M = 1.$$

Per il **il Principio Fondamentale**, il termine precedente, ovvero $a^{q \cdot 2^{i-1}} \bmod M$, deve essere 1 oppure -1. Infatti il principio dice che, se M è primo e $x^2 \equiv 1 \bmod M$, allora $x \equiv 1$ o $x \equiv -1$.

Essendo $a^{q \cdot 2^{i-1}}$ la radice di $a^{q \cdot 2^i}$, se quest'ultimo è congruo a 1 mod M , allora $a^{q \cdot 2^{i-1}} \equiv \pm 1 \pmod{M}$, ovvero $a^{q \cdot 2^{i-1}} \bmod M = \pm 1$.

Poiché partivamo dall'assunto che il termine con esponente i fosse il primo pari a 1, $a^{q \cdot 2^{i-1}}$ non può che valere -1. Allora

$$a^{q \cdot 2^{i-1}} \bmod M = -1, \text{ e quindi } a^{q \cdot 2^{i-1}} \bmod M = M-1.$$

Avendo trovato questo risultato, possiamo ora definire un

Algoritmo per il test di Miller-Rabin

find k and q such that $M-1 = 2^k \cdot q$

```

if  $a^q \bmod M = 1$  return("M probably prime") //caso 1, esito b
for j=0 to k-1 do //caso 2, cerco "l'ultimo" -1
if  $a^{q \cdot 2^j} \bmod M = M-1$  return("M probably prime") //esito b
return("M not prime") //esito a

```

3) Un algoritmo efficiente per generare una radice primitiva di q

Algoritmo per trovare una radice primitiva di q

1. generare $a < q$ a caso tale che a e q siano coprimi
2. fattorizzare $q-1$ in $f_1, f_2, \dots, f_n$
3. se esiste un f_i tale che $a^{(q-1)/f_i} \bmod q = 1$, occorre generare un nuovo a tornando al passo 1; altrimenti, restituire a.

Dimostrazione algoritmo radice primitiva:

Per definizione, se a è una radice primitiva di q, elevandola a esponenti i da 0 a q-1 e facendo modulo q del risultato, ottengo 1 soltanto quando $i = 0$ e sicuramente quando $i = q - 1$ (**P.T.d.F**), ovvero il ciclo si ripete dopo q-1. Se invece a NON è radice primitiva di q, esisterà un esponente $i < q - 1$ tale per cui il modulo q del risultato darà 1, e tale esponente sarà un divisore di q-1 per un certo k. Quindi i soli valori che danno resto 1 sono gli $a^{i \cdot k}$ per ogni i.

Dunque:

$$a^{q-1} \bmod q = a^{i \cdot k} \bmod q = 1$$

Di i e k esplicito i fattori primi:

$$(a^{i_1 \cdot i_2 \cdot \dots \cdot i_s})^{k_1 \cdot k_2 \cdot \dots \cdot k_t} \bmod q = 1$$

Ma so che:

$$a^i \bmod q = 1, \text{ ovvero } (a^{i_1 \cdot i_2 \cdot \dots \cdot i_s}) \bmod q = 1;$$

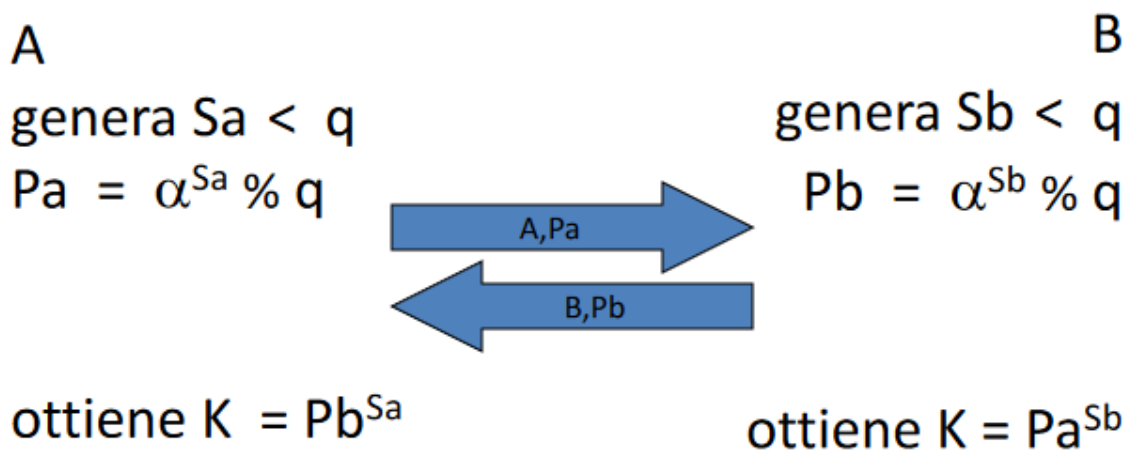
e abbiamo detto che $a^{i \cdot x} \bmod q = 1$ per ogni x, quindi anche per $x = k_1 \cdot k_2 \cdot \dots \cdot k_{t-1}$ (in tal caso, $i \cdot x = (q-1)/k_t$). Perciò

$$a^{i_1 \cdot i_2 \cdot \dots \cdot i_s \cdot k_1 \cdot k_2 \cdot \dots \cdot k_{t-1}} \bmod q = a^{(q-1)/k_t} \bmod q = 1.$$

cioè ipotizzando a NON radice primitiva, ho dimostrato che posso trovare un fattore di $(q - 1)$ tale che a elevata a esso dà 1 in modulo q , c.v.d.

II) Come funziona Diffie-Hellman?

1) Fasi di Diffie-Hellman



1. **A** e **B** generano a caso una chiave privata Sa e Sb , minori del q che è stato generato. Sia q che α sono parametri pubblici noti ad A e B, ed eventualmente a un avversario. Invece Sa e Sb sono segrete, generate casualmente e imprevedibilmente
2. **A** e **B** generano ciascuno la propria chiave pubblica, elevando rispettivamente a Sa e Sb la radice primitiva α nell'aritmetica modulo q
3. Per poter calcolare la chiave condivisa K , **A** deve elevare la chiave pubblica di **B** alla propria chiave privata, e **B** deve fare il viceversa. Dunque **A** manda a **B** la propria chiave pubblica, identificandosi in associazione con essa, e viceversa. Ciascuno memorizza tale associazione riguardante l'altro
4. A questo punto **A** ottiene K elevando alla propria chiave privata Sa quella pubblica Pb , e **B** viceversa, e **ottengono la stessa chiave K**.
5. A questo punto possono usare K per cifrare il traffico tra loro

1) Problemi di Diffie-Hellman

- Diffie-Hellman protegge da eavesdroppers, ma non da attacchi di tipo attivo: l'attacco **Man in the Middle** può falsificare le chiavi pubbliche inviate al punto 3. L'avversario **C** può generare a sua volta una chiave privata S_c e una pubblica P_c . Poi intercetta le chiavi pubbliche che **A** e **B** si scambiano; sostituisce il messaggio con il proprio, in cui si associa, identificandosi come il mittente legittimo, alla propria chiave pubblica P_c . Fa questo in un senso e in un altro, ovvero fa credere ad **A** che la chiave pubblica di **B** sia P_c e a **B** che la chiave pubblica di **A** sia P_c . **A** calcola la chiave condivisa usando quella che crede essere la chiave pubblica di **B**, ma in realtà è quella di **C**, e **B** fa lo stesso. **C** genera la chiave condivisa K_1 verso **A** usando la chiave pubblica di **A**; ovviamente a questo punto **B** genererà una chiave condivisa k_2 diversa da quella generata di **A**, e dunque K_1 e K_2 sono diverse: **A** e **B** non condividono la chiave. **C** invece ha una chiave condivisa con **A**, K_1 , e con **B**, K_2 . **C** se lo desidera può intercettare e decifrare con K_1 i messaggi da **A**, ricifrarli con K_2 e inoltrarli a **B**, che non si rende conto di nulla, e viceversa. Così **A** e **B** continuano a parlare ignari del **Man in the Middle**. Dunque DH permette di scambiare chiavi **solo assumendo che la chiave pubblica inviata all'interlocutore non possa essere falsificata**. La soluzione è l'utilizzo di un canale autenticato per lo scambio di chiavi pubbliche. Ci sono vari metodi per farlo, che si trattano in altri capitoli.
- Non è un vero e proprio cifrario, effettivamente non si cifra nulla; ci si scambia solamente chiavi. Con la chiave condivisa che si ottiene grazie a DH, che è simmetrica, si andrà poi a cifrare simmetricamente i messaggi. La cifratura asimmetrica diretta verrà realizzata in altri modi, ad esempio con RSA.